L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #1 Due: 2025-01-31

Learning Objectives

In this assignment you will demonstrate:

- ✅ Fluency in sketching **abstract syntax trees**
- ✅ The ability to craft a **grammar** from a natural language description of a programming language
- ✅ The ability to design and test a syntax with the **Ohm Editor**
- ✅ The creation of a **Node.js** project on GitHub, with a README, multiple imported packages, and 100% unit test coverage

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Go through Nicky Case's *How to Remember Everything Forever-ish*.
- ✅ Read Atul Gawande's *The Checklist*.
- ✅ Make a pass over Gabe Robins' *A Brief History of Computing*.
- ✅ Work through the course notes on Computation, Language Design, and Ohm.
- ✅ What all embedded videos in the course notes above. Pay particular attention to the videos by Bret Victor and Alan Kay, as these will give you the right mindset for designing your own language in this course.
- ✅ Read the Introduction and Chapters 1–2 of [Bernhardt].
- ✅ Read the Ohm README, the Ohm Tutorial, and the four pages of documentation (Syntax, API, Philosophy, and Patterns & Pitfalls) linked from the main Ohm Documentation.

Submission Instructions

All homework in this class is to be done in your three to six-person group. **DO NOT "partition" the work.** Letting a team member do some exercises while you do others cheapens your educational experience and leaves you behind. It is best for each student to do all of the exercises, then meet as a group to (1) select the "best" answer for submission when more than one exist, and (2) teach each other. If one group member is unable to do one of the exercises, another group member can teach that student how it do it. Everyone is responsible for understanding the entire homework submission, so everyone should contribute to the entire assignment.

Select one student to upload *the* single submission for the entire team.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student.
- ✅ For each student, an affidavit that each of the readings/watchings above were individually completed.
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project. This is to be a **public** GitHub repository named after the language you are creating. You will use this repo to host your compiler project, which will grow during each assignment. By the end of the term it will be amazing.

Add me as a committer to your repository. My github name is **rtoal**. This will help me help you during office hours if your project seems to be flailing.

Exercises

1. Give the AST for the following JavaScript program, using the level of detail we used during class. You can use my JS AST Viewer to guide you and check your work, but remember, the drawing you need to produce for full credit will be far less verbose than the tool's output. (Remember, the tool uses a third-party parser, esprima-next, that provides a complete ESTree-compliant AST, which is far more verbose than expected for hand-drawn ASTs.)

```
let [x, y] = [0, 0];
console.log(93.8 * 2 ** x + y);
```

2. Draw the AST for the following JavaScript program, using the level of detail we used during class.

```
import x from "x"
```

```
console.log(93.8 * {x} << x.r[z])
```

3. Draw the AST for the following JavaScript program, using the level of detail we used during class.

```
const x = x / {[x]: `${x}`}[x]("y")
```

4. For the following JavaScript fragment (not a complete program), draw the AST, using the level of detail we used during class.

```
class C {
  f({a, b: c}) {return ([a,f]) => C}
}
```

5. For each scenario below, give Ohm grammar rules for `Exp`, `Term`, `Factor`, and `Primary` that make the expression `-2**2`:
   a. Evaluate to 4.
   b. Evaluate to `-4`.
   c. Be a syntax error, while allowing `(-2)**2` and `-(2**2)` to be legal.

6. Astro is a really tiny language, so we'd like to make Astro++. This new language adds an if-statement, a while statement, a break statement, and relational operators. The while statement should start with the keyword `while`, followed by a test expression, followed by a block (a curly-brace delimited sequence of statements). The if-statement should start with the keyword `if`, followed by a test expression, then a block, then an optional else-part which is the keyword `else` followed by either a block or another if-statement. Neither the while statement nor the if statement should end with a semicolon. The break statement is only allowed to appear in a while statement's block. The relational operators are the same as those in Python and are to be NON-associative. All of the relational operators are on the same precedence level, lower than all other operators. Give the syntax of Astro++ using Ohm. Hint: Check your grammar with the Ohm Editor so that you don't needlessly throw away points.

For Your Project

In this class you will not only be writing a compiler—you will be writing a compiler for a language that you *design yourself*. You have wide latitude in designing your language: I want you to design a language you can actually love. The only requirements are: (1) it must be of sufficient complexity (i.e., enough interesting features, including features beyond the example languages we see in class), and (2) it requires significant

work in the static analysis phase. You can satisfy the latter requirement with a statically-typed language with static scoping (in addition to a few other checks of course), but it *may* be possible to design a dynamically typed language, provided you define a significant number of statically checked contextual constraints. If you go with a dynamically-typed language, you should check with me first to make sure your workload is fair.

- Complete the first pass of the design of your language. USE THE OHM EDITOR. Give your language description on the README.md file of a public GitHub repository you will be setting up for the project. You should have, in your README:

  - ✅ A nice logo (please put some care into this)
  - ✅ The name of your language, in a large font (unless the language name is readable from the image)
  - ✅ A one paragraph introduction (tell a story!)
  - ✅ A list of features
  - ✅ Lots of example programs. You can do the "your language on the left, JavaScript on the right" presentation style.

  A list of previous projects from students that have taken this class over the last decade or so can be found in the Course Notes on Language Design. You may find these helpful, but do beware that some of them may be lacking in some areas.

- Set up your project. Include a *package.json* which is nicely filled out with at least the proper project name and the names of all the team members in the `author` field. Add the property `"type": "module"` to allow for modern JavaScript. Ensure that `npm test` runs tests with coverage. For now, just fill the examples folder with a few example programs—at least five significant ones. You don't have to start writing the compiler, but since you want to make sure your project does *something*, make your main file simply print "Hello, world" or something silly or fun, but not the Bee Movie script or a Rickroll, as those are not yet old enough to be retro. Your test program needs at least one test, it can be as simple as testing that 1 = 1.

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json        -- optional, you don't have to have one
├── docs
|   └── logo.png
├── examples
|   └── ...                 -- at least 5 significant examples
├── src
|   └── (yourlangname).js   -- Just print hello, world
```

```
└── test
    └── compiler.test.js    -- Just assert 1===1 for now
```

Grading Rubric

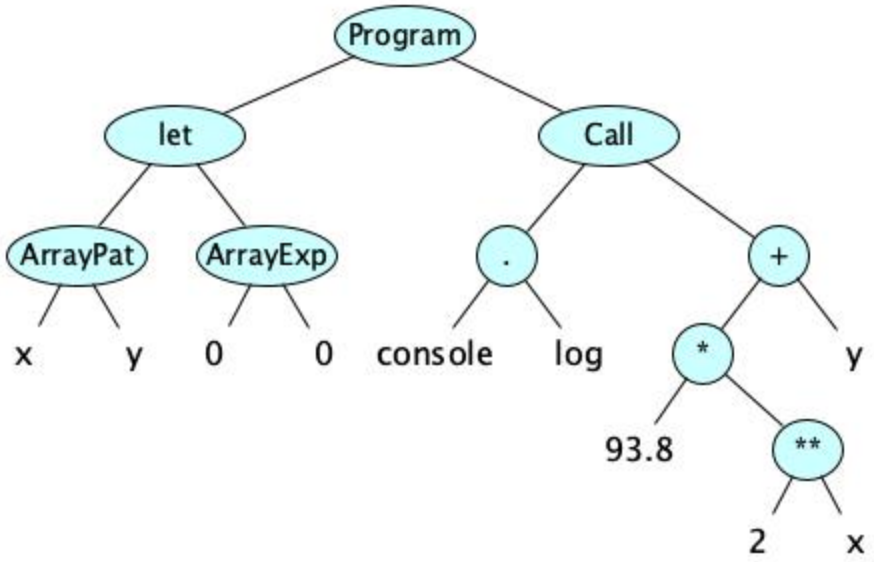
To help you get a good score, here are all the things you will be graded on.

- Exercises (74 pts)
  - ✅ Exercise 1 (12 pts)
  - ✅ Exercise 2 (12 pts)
  - ✅ Exercise 3 (12 pts)
  - ✅ Exercise 4 (12 pts)
  - ✅ Exercise 5 (12 pts)
  - ✅ Exercise 6 (14 pts)
- Compiler Project (26 pts)
  - ✅ Project GitHub repo has same name as language (2 pts)
  - ✅ Project name appears in README (2 pts)
  - ✅ Logo appears in README (2 pts)
  - ✅ README has story of your language (2 pts)
  - ✅ README lists features of the language (2 pts)
  - ✅ README shows examples of programs (2 pts)
  - ✅ package.json contains all of the team members in the author property (2 pts)
  - ✅ package.json has "type": "module" (2 pts)
  - ✅ npm test will run, execute at least one test, and show 100% test coverage (2 pts)
  - ✅ Examples folder has at least 5 full example programs and covers the language features (2 pts)
  - ✅ Structure of project is exactly as specified on this page (2 pts)
  - ✅ .gitignore is present in repo and properly generated (let Github do it for you) (2 pts)
  - ✅ No extra stuff in repo (no node_modules, no .DS_Store, etc.) (2 pts)

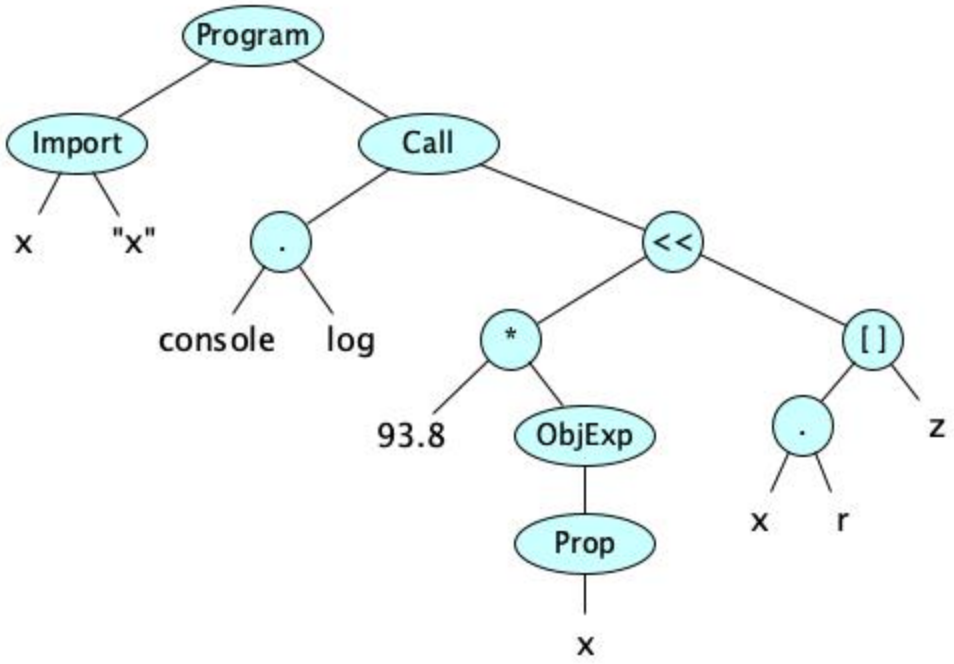
LANGUAGES AND AUTOMATA II

HOMEWORK #1 PARTIAL ANSWERS

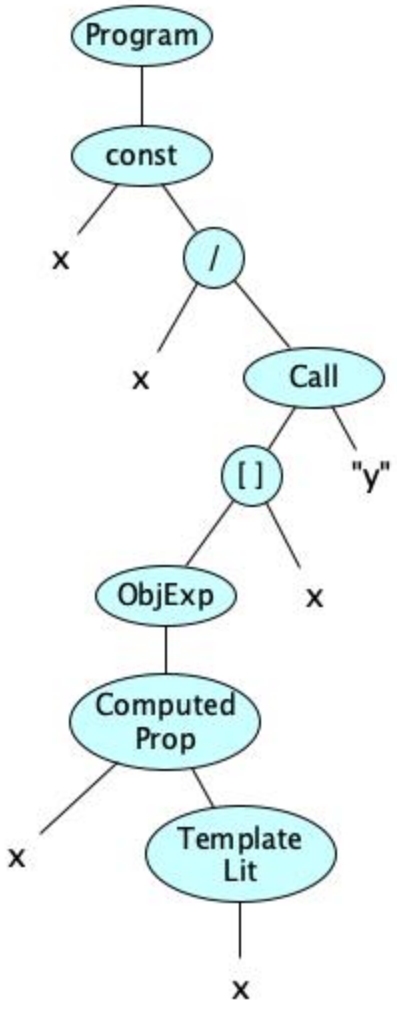
1.
```
let [x, y] = [0, 0];
console.log(93.8 * 2 ** x + y);
```



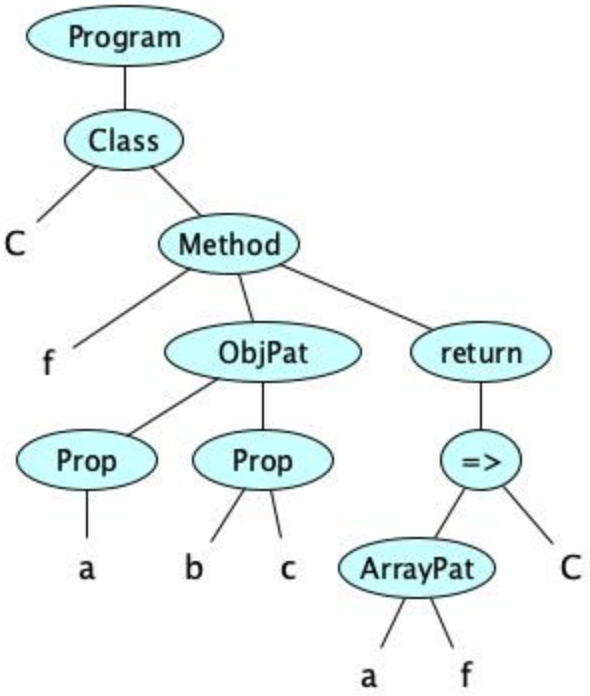
2.
```
import x from "x"
console.log(93.8 * {x} << x.r[z])
```



3. `const x = x / {[x]: `${x}`}[x]("y")`



4.
```
class C {
  f({a, b: c}) {return ([a, f]) => C}
}
```



5. To make the expression `-2**2` ...

   a. ...evaluate to 4, we make negation have higher precedence than exponentiation:

```
Exp     = Exp "+" Term | Term
Term    = Term "*" Factor | Factor
Factor  = Primary "**" Factor | Primary
Primary = "-" Primary | num | id | "(" Exp ")"
```

   b. ...evaluate to -4, we make negation have lower precedence than exponentiation. You can put the negation at the beginning of wither the `Exp` or the `Term` rule:

```
Exp     = "-"? (Exp "+" Term | Term)
Term    = Term "*" Factor | Factor
Factor  = Primary "**" Factor | Primary
Primary = num | id | "(" Exp ")"
```

   c. ...be a syntax error, we put negation and exponentiation in the same rule:

```
Exp     = Exp "+" Term | Term
Term    = Term "*" Factor | Factor
```

```
  Factor  = Primary "**" Factor | "-"? Primary
  Primary = num | id | "(" Exp ")"
```

6. Here is a grammar for Astro++ that enforces the appearance of break statements only within while loops. (In practice, most people don't actually capture this in the grammar, but it's a good exercise to actually do it!)

```
Astro {
  Program       = StmtNoBreak+
  StmtNoBreak   = Assignment
                | PrintStmt
                | IfStmtNoBreak
                | WhileStmt
  Stmt          = Assignment
                | PrintStmt
                | IfStmt
                | WhileStmt
                | BreakStmt
  Assignment    = id "=" Exp ";"
  PrintStmt     = print Exp ";"
  BreakStmt     = break ";"
  WhileStmt     = while Exp Block
  IfStmtNoBreak = if Exp BlockNoBreak (else (BlockNoBreak|IfStmtNoBreak))?
  IfStmt        = if Exp Block (else (Block|IfStmt))?
  BlockNoBreak  = "{" StmtNoBreak* "}"
  Block         = "{" Stmt* "}"
  Exp           = Exp1 ("<="|"<"|"=="|"!="|">="|">") Exp1  --relational
                | Exp1
  Exp1          = Exp1 ("+" | "-") Term                    --additive
                | Term
  Term          = Term ("*" | "/" | "%") Factor            --multiplicative
                | Factor
  Factor        = Primary "**" Factor                      --exponentiation
                | "-" Primary                              --negation
                | Primary
  Primary       = id "(" ListOf<Exp, ","> ")"              --call
                | numeral                                  --num
                | id                                       --id
                | "(" Exp ")"                              --parens
  numeral       = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  print         = "print" ~idchar
  while         = "while" ~idchar
  if            = "if" ~idchar
  else          = "else" ~idchar
  break         = "break" ~idchar
```

```
  keyword       = print | while | if | else | break
  idchar        = letter | digit | "_"
  id            = ~keyword letter idchar*
  space        += "//" (~"\n" any)*                    --comment
}
```

L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #2 Due: 2025-02-21

Learning Objectives

In this assignment you will demonstrate:

- ✅ The ability to craft a language grammar from a natural language description of a programming language
- ✅ The ability to write parsers using the Ohm library
- ✅ The ability to write test cases for a parser
- ✅ The ability to extend a Node.js project on GitHub while maintaining 100% unit test coverage

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Do any of the assigned readings and watchings for Homework 1 that you did not get to before.
- ✅ Work through the course notes on Astro, Bella, Carlos, Compilers and Interpreters, and How to Write a Compiler (up to the section entitled *Design the Program Representation*).
- ✅ Read Chapters 1–1.4 of [Mogenson].
- ✅ Read Ch. 3–5 of [Bernhardt].
- ✅ Re-read (or Skim) the Ohm README, the Ohm Tutorial, and the four pages of documentation (Syntax, API, Philosophy, and Patterns & Pitfalls) linked from the main Ohm Documentation.
- ✅ Skim Ch. 2.1–2.5, 2.10, and 3.1–3.6 of [Mogenson]
- ✅ The Wikipedia page on Parsing Expression Grammars.
- ✅ Watch any videos embedded in the relevant course notes.

Submission Instructions

As usual, turn in a single submission for your entire group via BrightSpace. Remember to not simply partition the work among team members, as this reduces your learning.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project

Exercises

1. Give tiny little Ohm grammars for:
   a. Canadian Postal Codes (make sure to prohibit D, F, I, O, Q, U)
   b. Legal Visa® Card Numbers, *ignoring the Luhn checksums*, i.e., accept `4` + 15 digits or `4` + 12 digits.
   c. Legal MasterCard® Numbers, *ignoring the Luhn checksums*, i.e., accept `51–55` + 14 digits or `2221–2720` + 12 digits.
   d. Strings of Basic Latin letters **except** those strings that are exactly three letters ending with two Latin letter o's, of any case.
   e. Binary numerals divisible by 16.
   f. Decimal numerals in the range 8 through 32, inclusive.
   g. All strings of Unicode letters, except `python`, `pycharm`, or `pyc`.
   h. Floating point constants that are allowed to have an empty fractional part, but whose exponent part is required and can have no more than three digits in the exponent part
   i. Palindromes over the letters a, b, and c, of length 2, 3, 5, or 8
   j. Python string literals. Don't get too fancy here—just translate what you see in the Python Reference linked above into Ohm notation.

   Feel free to use the following template, which comes with some unit tests:

```
import { describe, it } from "node:test"
import assert from "assert"
import * as ohm from "ohm-js"

const grammars = {
  canadianPostalCode: String.raw`
```

```
    write your grammar here
  `,

  visa: String.raw`
    write your grammar here
  `,

  masterCard: String.raw`
    write your grammar here
  `,

  notThreeEndingInOO: String.raw`
    write your grammar here
  `,

  divisibleBy16: String.raw`
    write your grammar here
  `,

  eightThroughThirtyTwo: String.raw`
    write your grammar here
  `,

  notPythonPycharmPyc: String.raw`
    write your grammar here
  `,

  restrictedFloats: String.raw`
    write your grammar here
  `,

  palindromes2358: String.raw`
    write your grammar here
  `,

  pythonStringLiterals: String.raw`
    write your grammar here
  `,
}

function matches(name, string) {
  const grammar = `G {${grammars[name]}}`
```

```
  return ohm.grammar(grammar).match(string).succeeded()
}

const testFixture = {
  canadianPostalCode: {
    good: ["A7X 2P8", "P8E 4R2", "K1V 9P2", "Y3J 5C0"],
    bad: [
      "A7X   9B2",
      "C7E 9U2",
      "",
      "Dog",
      "K1V\t9P2",
      " A7X 2P8",
      "A7X 2P8 ",
    ],
  },
  visa: {
    good: ["4128976567772613", "4089655522138888", "4098562516243"],
    bad: [
      "43333",
      "42346238746283746823",
      "7687777777263211",
      "foo",
      "π",
      "4128976567772613 ",
    ],
  },
  masterCard: {
    good: [
      "5100000000000000",
      "5294837679998888",
      "5309888182838282",
      "5599999999999999",
      "2221000000000000",
      "2720999999999999",
      "2578930481258783",
      "2230000000000000",
    ],
    bad: [
      "5763777373890002",
      "513988843211541",
      "51398884321108541",
```

```
    "",
    "OH",
    "5432333xxxxxxxxx",
  ],
},
notThreeEndingInOO: {
  good: ["", "fog", "Tho", "one", "a", "ab", "food"],
  bad: ["fOo", "gOO", "HoO", "zoo", "MOO", "123", "A15"],
},
divisibleBy16: {
  good: [
    "0",
    "00",
    "000",
    "00000",
    "00000",
    "000000",
    "00000000",
    "1101000000",
  ],
  bad: ["1", "00000000100", "1000000001", "dog0000000"],
},
eightThroughThirtyTwo: {
  good: Array(25)
    .fill(0)
    .map((x, i) => i + 8),
  bad: ["1", "0", "00003", "dog", "", "361", "90", "7", "-11"],
},
notPythonPycharmPyc: {
  good: [
    "",
    "pythons",
    "pycs",
    "PYC",
    "apycharm",
    "zpyc",
    "dog",
    "pythonpyc",
  ],
  bad: ["python", "pycharm", "pyc"],
},
restrictedFloats: {
```

```
  good: ["1e0", "235e9", "1.0e1", "1.0e+122", "55e20"],
  bad: ["3.5E9999", "2.355e-9991", "1e2210"],
},
palindromes2358: {
  good: [
    "aa",
    "bb",
    "cc",
    "aaa",
    "aba",
    "aca",
    "bab",
    "bbb",
    "ababa",
    "abcba",
    "aaaaaaaa",
    "abaaaaba",
    "cbcbbcbc",
    "caaaaaac",
  ],
  bad: ["", "a", "ab", "abc", "abbbb", "cbcbcbcb"],
},
pythonStringLiterals: {
  good: String.raw`''
    ""
    'hello'
    "world"
    'a\'b'
    "a\"b"
    '\n'
    "a\tb"
    f'\u'
    """abc"""
    '''a''''''''
    """abc\xdef"""
    '''abc\$def'''
    '''abc\''''`
    .split("\n")
    .map((s) => s.trim()),
  bad: String.raw`
    'hello"
    "world'
```

```
        'a'b'
        "a"b"
        'a''
        "x""
        """""""""
        frr"abc"
        'a\'
        '''abc''''
        """`
        .split("\n")
        .map((s) => s.trim()),
    },
  }

  for (let name of Object.keys(testFixture)) {
    describe(`when matching ${name}`, () => {
      for (let s of testFixture[name].good) {
        it(`accepts ${s}`, () => {
          assert.ok(matches(name, s))
        })
      }
      for (let s of testFixture[name].bad) {
        it(`rejects ${s}`, () => {
          assert.ok(!matches(name, s))
        })
      }
    })
  }
```

2. Here is a description of a language. Programs in this language are made up of a possibly empty sequence of function declarations, followed by a single expression. Each function declaration starts with the keyword `func` followed by the function's name (an identifier), then a parenthesized list of zero or more parameters (also identifiers) separated by commas, then the body, which is a sequence of one or more expressions separated (NOT terminated) by semicolons with the expression sequence terminated with the keyword `end`. Expressions can be numeric literals, string literals, identifiers, function calls, or can be made up of other expressions with the usual binary arithmetic operators (plus, minus, times, divide) and a unary prefix negation and a unary postfix factorial (`!`). There's a conditional expression with the syntax `y if x else z`. Factorial has the highest precedence, followed by negation, the multiplicative operators, the additive operators, and finally the conditional. Parentheses are used, as in most other languages, to group subexpressions. Numeric literals are non-empty sequences of decimal digits with an optional fractional part and an optional exponent part.

String literals delimited with double quotes with the escape sequences `\'`, `\"`, `\n`, `\\`, and `\u{` *hhhhhh* `}` where *hhhhhh* is a sequence of one-to-six hexadecimal digits. Identifiers are non-empty sequences of letters, decimal digits, underscores, at-signs, and dollar signs, beginning with a letter or at-sign, that are not also reserved words. Function calls are formed with an identifier followed by a comma-separated list of expressions bracketed by square brackets. Comments are `--` until the end of the line.

Write a single example program that covers every aspect of this definition.

3. For the language described in the previous exercise, write a complete syntactic description of this language in Ohm. (Hint: use the Ohm editor to check your work. Use your everything-program from the previous exercise as a positive test, but write a few negative tests, too, so that you can check that your grammar does not "match too much." When grading, I will copy-paste your submitted solution into the Ohm editor with my own detailed test suite).

For Your Project

Continue your compiler project in the public GitHub repository you created in the first assignment. You will be expanding your repo to have the following:

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json
├── docs
|   └── ...                 -- image file for your logo
├── examples
|   └── ...                 -- lots of example programs
├── src
|   ├── (yourlanguagename).js
|   ├── (yourlanguagename).ohm
|   ├── compiler.js         -- copied from the course notes
|   ├── core.js             -- this will be empty for now
|   ├── parser.js           -- copied from the course notes
|   ├── analyzer.js         -- contains just the stub function
|   ├── optimizer.js        -- contains just the stub function
|   └── generator.js        -- contains just the stub function
└── test
    ├── compiler.test.js    -- just test that you can invoke the parser
    └── parser.test.js      -- new, test Ohm parsing here
```

Your tasks for this assignment are the following:

- Expand your README as required.
- Construct the files (yourprojectname).js, compiler.js, (yourprojectname).ohm, core.js, analyzer.js, grammar.test.js, and analyzer.test.js as instructed in the How to Write a Compiler notes.
- Design and implement the syntax of your language in an Ohm grammar in your `.ohm` file.
- In your parser.js file, write a parser...a function that will return the Ohm grammar match.
- For the analyzer, optimizer, and generator, though, since these parts are to be written future assignments, you will simply implement them as follows:

```
export default function analyze() {
  throw new Error("Not yet implemented")
}


export default function optimize() {
  throw new Error("Not yet implemented")
}


export default function generate() {
  throw new Error("Not yet implemented")
}
```

- Write parser tests.
- Ensure that Running

```
node src/(yourlanguagename).js (filename) parsed
```

  either produces `Syntax is ok` or displays an error.

- Ensure that running

```
npm test
```

  runs all your tests, that they all pass, and the coverage report shows 100% coverage.

Grading Rubric

To help you get a good score, here are all the things you will be graded on.

- Exercises (60 pts)

- ✅ Exercise 1 (30 pts)
- ✅ Exercise 2 (20 pts)
- ✅ Exercise 3 (10 pts)

- Compiler Project (40 pts)
  - ✅ Project GitHub repo is in order, exactly as specified above (3 pts)
  - ✅ GitHub repo has no additional files (e.g., no node_modules, no .DS_Store, etc.) (1 pt)
  - ✅ GitHub repo has a README with language name, title, and good description (2 pts)
  - ✅ README has the names of all students (2 pts)
  - ✅ Your project can be cloned (1 pt)
  - ✅ I can run npm test immediately after cloning (1 pt)
  - ✅ All tests pass (3 pts)
  - ✅ Tests have great coverage (you should be able to get 100%, accept nothing less) (2 pts)
  - ✅ The Ohm specification is great and complete (20 pts)
  - ✅ Running from the command line will give the correct answer to the syntax check (5 pts)

LANGUAGES AND AUTOMATA II

HOMEWORK #2 PARTIAL ANSWERS

1. Ohm Exercises:

ohm_exercises.js

```
import * as ohm from "ohm-js"

const grammars = {
  canadianPostalCode: String.raw`
    code  = alpha digit alpha " " digit alpha digit
    alpha = "A".."C" | "E" | "G".."H" | "J".."N" | "P" | "R".."T" | "V".."Z"
  `,

  visa: String.raw`
    card = "4" d d d d d d d d d d d d d (d d d)?
    d    = digit
  `,

  masterCard: String.raw`
    card = "5" "1".."5" d d d12     --fives
         | "222" "1".."9" d12       --range2221_2229
         | "22" "3".."9" d d12      --range2230_2299
         | "2" "3".."6" d d d12     --range2300_2699
         | "27" "0".."1" d d d12    --range2700_2719
         | "2720" d12               --range2720
    d12  = d d d d d d d d d d d d
    d    = digit
  `,

  notThreeEndingInOO: String.raw`
    ok = long | threeWithNoFinal0 | threeWithNoPenultimate0 | two | char?
```

```
  long = char char char char+
  threeWithNoFinalO = char char nonOChar
  threeWithNoPenultimateO = char nonOChar char
  two = char char
  char = "A".."Z" | "a".."z"
  nonOChar = ~"o" ~"O" char
`,

divisibleBy16: String.raw`
  ok = (~("0000" end) ("0"|"1"))* "0000"   -- nonzero
     | "0" "0"? "0"? "0"?                  -- zero
`,

eightThroughThirtyTwo: String.raw`
  ok = "1".."2" digit   -- tenToTwentyNine
     | "3" "0".."2"     -- thirtyToThirtyTwo
     | "8".."9"         -- eightToNine
`,

notPythonPycharmPyc: String.raw`
  word = ~("python" end) ~("pycharm" end) ~("pyc" end) letter*
`,

restrictedFloats: String.raw`
  float = digit+ frac? exp
  frac = "." digit+
  exp = ("E" | "e") ("+" | "-")? digit digit? digit?
`,

palindromes2358: String.raw`
  pal2358 = pal8 | pal5 | pal3 | pal2
  pal8    = "a" pal6 "a"
          | "b" pal6 "b"
          | "c" pal6 "c"
  pal6    = "a" pal4 "a"
          | "b" pal4 "b"
          | "c" pal4 "c"
  pal5    = "a" pal3 "a"
          | "b" pal3 "b"
```

```
            | "c" pal3 "c"
    pal4    = "a" pal2 "a"
            | "b" pal2 "b"
            | "c" pal2 "c"
    pal3    = "a" pal1 "a"
            | "b" pal1 "b"
            | "c" pal1 "c"
    pal2    = "aa" | "bb" | "cc"
    pal1    = "a" | "b" | "c"
  `,

  pythonStringLiterals: String.raw`
    stringliteral     =  stringprefix? (longstring | shortstring)
    stringprefix      =  "r" | "u" | "R" | "U" | "f" | "F"
                      |  "fr" | "Fr" | "fR" | "FR" | "rf" | "rF" | "Rf" | "RF
    shortstring       =  "'" shortstringitem_s* "'" | "\"" shortstringitem_d*
    longstring        =  "'''" longstringitem_s* "'''" | "\"\"\"" longstringi
    shortstringitem_s =  shortstringchar_s | stringescapeseq
    shortstringitem_d =  shortstringchar_d | stringescapeseq
    longstringitem_s  =  longstringchar_s | stringescapeseq
    longstringitem_d  =  longstringchar_d | stringescapeseq
    shortstringchar_s =  ~"\n" ~"\\" ~"'" any
    shortstringchar_d =  ~"\n" ~"\\" ~"\"" any
    longstringchar_s  =  ~"\\" ~"'''" any
    longstringchar_d  =  ~"\\" ~"\"\"\"" any
    stringescapeseq   =  "\\" any
  `,
}

export function matches(name, string) {
  const grammar = `G {${grammars[name]}}`
  return ohm.grammar(grammar).match(string).succeeded()
}
```

2. Answers vary. You just need to cover *every* rule.

3. An Ohm Grammar for a Mystery Language:

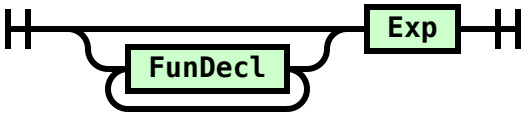
```
Mystery {
  Program   = FunDecl* Exp
  Exp       = Exp1 if Exp1 else Exp     --conditional
            | Exp1
  Exp1      = Exp1 ("+" | "-") Exp2     --additive
            | Exp2
  Exp2      = Exp2 ("*" | "/") Exp3     --multiplicative
            | Exp3
  Exp3      = "-" Exp4                  --negation
            | Exp4
  Exp4      = Exp5 "!"                  --factorial
            | Exp5
  Exp5      = num
            | str
            | Call
            | id
            | "(" Exp ")"               --parens
  FunDecl   = func id Params Body
  Params    = "(" ListOf<id, ","> ")"
  Body      = NonemptyListOf<Exp, ";"> endkw
  Call      = id Args
  Args      = "[" ListOf<Exp, ","> "]"

  id        = ~keyword (letter | "@") idchar*
  idchar    = alnum | "_" | "@" | "$"
  endkw     = "end" ~idchar
  func      = "func" ~idchar
  if        = "if" ~idchar
  else      = "else" ~idchar
  keyword   = endkw | func | if | else
  num       = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  str       = "\"" char* "\""
  char      = ~("\"" | "\\") any       --direct
            | "\\" escape               --escaped
  escape    = "'" | "\"" | "\\" | "n"
            | "u{" hexes "}"            --codepoint
  hexes     = h h? h? h? h? h?
  h         = hexDigit
```

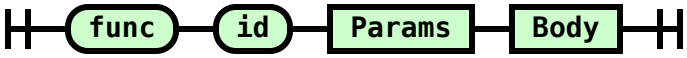
```
    space    += "--" (~"\n" any)*          --comment
  }
```

BONUS: Here are the syntax diagrams for the phrase categories of the language above:

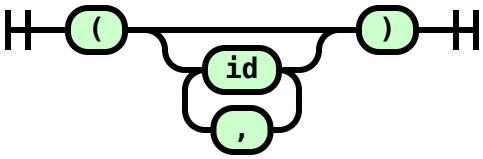
Program



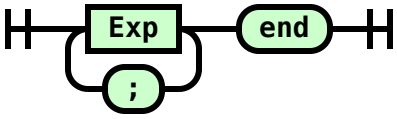
FunDecl



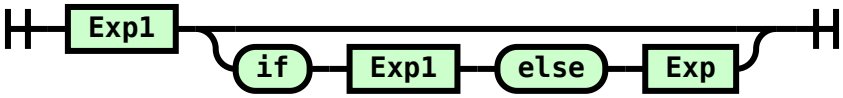
Params



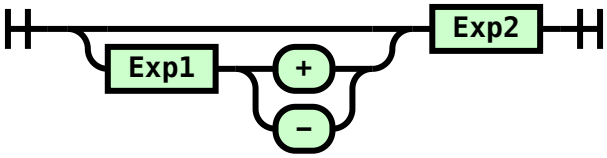
Body



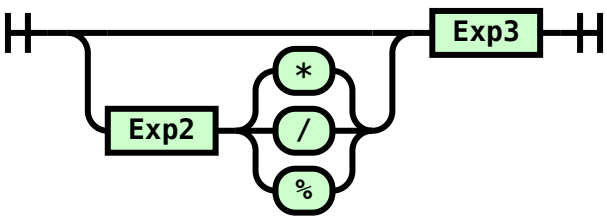
Exp



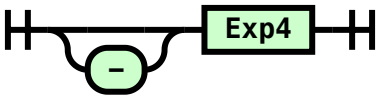
Exp1



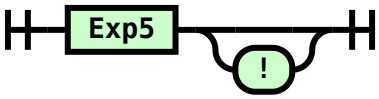
Exp2



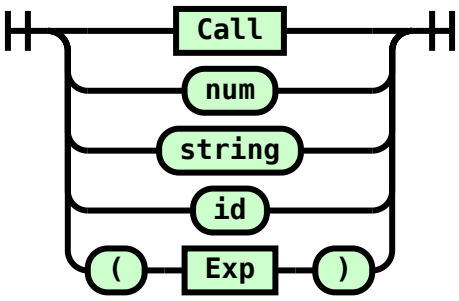
Exp3



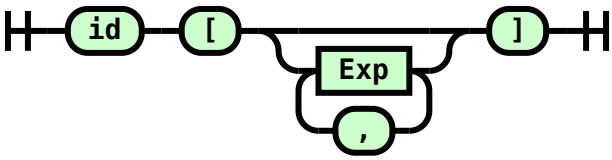
Exp4



Exp4



Call



L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #3 Due: 2025-03-30

Learning Objectives

In this assignment you will demonstrate:

- ✅ The ability to write a semantic analyzer for a mid-sized, statically-typed language
- ✅ Knowledge of static analysis in programming language theory
- ✅ Basic familiarity with types, functions, and concurrency
- ✅ The ability to distinguish different types of language-processing errors (lexical, context-free, static contextual, runtime)
- ✅ Competency in growing and maintaining a large Node.js project
- ✅ The ability to deploy a website hosted on GitHub Pages

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Do any of the assigned readings and watchings from the first two assignments that you did not get to.
- ✅ Work through the course notes on How to Write a Compiler (up to the section entitled *Optimization*).
- ✅ Read Chapters 4—6 of [Mogenson]
- ✅ Read this entire overview of Java Language changes from versions 9 through 23. The purpose of this reading is to get you familiar with modern programming languages features. There is a lot to this, so skim when it gets too detailed.
- ✅ Skim The Wikipedia page on Static Program Analysis
- ✅ Skim the OWASP brief article on Static Code Analysis (from a security perspective)
- ✅ Watch This video (you can play at 1.5x)
- ✅ Watch any videos embedded in the relevant course notes, as time allows

Submission Instructions

As usual, turn in a single submission for your entire group via BrightSpace. Remember to not simply partition the work among team members, as this reduces your learning.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project

Exercises

1. Classify the following as a syntax error, static semantic (contextual) error, or not a compile time error. In the case where code is given, assume all identifiers are declared, have the expected type, and are in scope. All items refer to the Java language.

    a. `x+++-y`

    b. `x---+y`

    c. incrementing a read-only variable

    d. code in class C accessing a private field from class D

    e. Using an uninitialized variable

    f. Dereferencing a null reference

    g. `null instanceof C`

    h. `!!x`

    i. `x > y > z`

    j. `if (a instanceof Dog d) {...}`

    k. `var s = """This is weird""";`

    l. `switch = 200;`

    m. `x = switch (e) {case 1->5; default->8;};`

2. How do JavaScript and Rust treat the following:

```
let x = 3;
let x = 3;
```

3. Describe how the languages Java and Ruby differ in their interpretations of the meaning of the keyword `private`. You can use an AI chatbot for help, but please trim down the long-winded applications those tools are known for, and give a concise explanation that proves you truly understand the difference.

4. Some languages do not require the parameters to a function call to be evaluated in any particular order. Is it possible that different evaluation orders can lead to different arguments being passed? If so, give an example to illustrate this point, and if not, prove that no such event could occur.

5. Describe in your own words how the Carlos language allows handles recursive structs. Describe what kinds of restrictions the language definition imposes and why. Describe how the compiler enforces the restrictions. Write well. Use technical vocabulary accurately. An AI assistant can help you get your grammar and spelling right, though it is unlikely to get the right answer.

6. Some languages do not have loops. Write a function, using tail recursion (and no loops) to compute the minimum value of an array or list in Python, C, JavaScript, and in either Go, Erlang, or Rust (your choice). Obviously these languages probably already have a min-value-in-array function in a standard library, but the purpose of this exercise is for you to demonstrate your understanding of tail recursion. Your solution must be in the classic functional programming style, that is, it must be stateless. Use parameters, not nonlocal variables, to accumulate values. Assume the array or list contains floating-point values.

7. Your friend creates a little JavaScript function to implement a count down, like so:

```JS
function countDownFrom10() {
  let i = 10;
  function update() {
    document.getElementById("t").innerHTML = i;
    if (i-- > 0) setTimeout(update, 1000);
  }
  update();
}
```

Your other friend says “Yikes, you are updating a non-local variable! Here is a better way:”

```JS
function countDownFromTen() {
  function update(i) {
```

```
    document.getElementById("t").innerHTML = i;
    if (i-- > 0) setTimeout(update(i), 1000);
  }
  update(10);
}
```

What does your second friend's function do when called? Why does it fail? Your friend is on the right path though. Fix their code and explain why your fix works.

8. Find as many linter errors as you can in this Java source code file (C.java):



```
import java.util.HashMap;

class C {
    static final HashMap<String, Integer> m = new HashMap<String, Integer>();

    static int zero() {
        return 0;
    }

    public C() {
    }
}
```

You can use SonarLint or FindBugs or FindSecBugs or PMD or whatever you prefer. You might even need to use a combination of tools because it is possible no tool finds them all. (Please note you are not expected to already know what all the issues are here. The idea is to practice with tools and have good discussions with teammates. Find as many as you can, and read and understand each problem that is reported to you so you learn (1) what kinds of potential bugs and security problems can exist even in compilable and runnable code, and (2) the kinds of things that a static analyzer *can* detect.)

For Your Project

Continue your compiler project in the public GitHub repository you created in the first assignment. You will be expanding your repo to have the following:

```
.
├── .gitignore
├── README.md
├── LICENSE
```

```
├── package.json
├── .prettierrc.json
├── docs
|   └── ...                 -- now with the companion website
├── examples
|   └── ...                 -- add more example programs
├── src
|   ├── (yourlanguagename).js
|   ├── (yourlanguagename).ohm
|   ├── compiler.js
|   ├── core.js             -- new!
|   ├── parser.js           -- as before
|   ├── analyzer.js         -- with a completed static analyzer!
|   ├── optimizer.js        -- still just the stub function
|   └── generator.js        -- still just the stub function
└── test
    ├── compiler.test.js    -- just test that you can parse and analyze
    ├── parser.test.js      -- as before, but improve from HW2 feedback
    └── analyzer.test.js    -- new!
```

Your tasks for this assignment are twofold. First, you are to implement the static analyzer for your compiler. Second, you are to start a companion website for your language.

- Implement the static analyzer in your analyzer.js file. Since you're using Ohm, the analysis will be overlaid into the existing Ohm semantics object that you wrote to prepare the AST in the previous assignment. For this assignment, you'll be simply enhancing this object. Here are a few things to be thinking about for your analyzer:

  - Scope resolution
  - Proper contextual use of `return`, `break`, etc.
  - Type checking
  - Type inference
  - Parameter matching
  - Pattern exhaustiveness
  - Access controls

  Make sure your language has a non-trivial number of contextual rules to be enforced at compile time. Generally speaking, your language should feature static types. If it does not, make up for the lack of type checking with something really special, like really a really sophisticated concurrency mechanism that will require more sophisticated code generation and optimization in later assignments.

- Make sure your README file talks about the required static checking. (If your language is a "dynamic" language without static checks, at least list which checks are being deferred to run time.)

- Add tests to the analyzer.test.js file. Follow the style of testing from the Bella and Carlos compilers. You need checks that show that contextually correct programs generate the proper representation, and

that all form of static errors are caught.

- For your language website (a.k.a. a “Home Page”) for your programming language using GitHub Pages. The site’s page (or pages) should consist of a single page and:
  - Be rather pretty, CSS-wise (you can use a template, of course).
  - Tell the story of your language in no more than three paragraphs.
  - Include examples of programs in your language. You need at least five complete, semantically correct, example programs, that cover every syntactic form of your language, and most if not all of the interesting semantic-level checks.
  - Include developer bios (and optionally pictures), because that’s kind of fun.
  - Include a link to the actual GitHub repo of the compiler.
  - Also add a link to this site from the README of the GitHub repo.

Grading Rubric

To help you get a good score, here are all the things you will be graded on.

- Exercises (48 pts)
  - ✅ Exercise 1 (13 pts)
  - ✅ Exercise 2 (5 pts)
  - ✅ Exercise 3 (5 pts)
  - ✅ Exercise 4 (5 pts)
  - ✅ Exercise 5 (5 pts)
  - ✅ Exercise 6 (5 pts)
  - ✅ Exercise 7 (5 pts)
  - ✅ Exercise 8 (5 pts)
- Compiler Project (42 pts)
  - ✅ Your project can be cloned (1 pt)
  - ✅ In the project README, there is a link to your GitHub Pages site (1 pt)
  - ✅ I can run `npm test` immediately after cloning (1 pt)
  - ✅ All tests pass, and there are enough of them (3 pts)
  - ✅ The context is managed well—preferably in an object (7 pts)
  - ✅ The error handling is well-structured—nice check functions or methods (5 pts)
  - ✅ At least five interesting static checks (7 pts)
  - ✅ Analysis method are written like a pro—they work, and well named, etc. (6 pts)
  - ✅ You have at least 50 tests in your suite (5 pts, all or nothing)
  - ✅ The list of static errors are on the README or on your language website (3 pts)
  - ✅ Test coverage is 100% (3 pts, all or nothing)
- Project Website (10 pts)

- ✅ The GH Pages site looks great: nice template, or hand-crafted CSS (3 pts)
- ✅ Site has the story of your language in no more than three paragraphs (2 pts)
- ✅ Site shows at least fives example programs your language (2 pts)
- ✅ Site has developer names and short bios (2 pts)
- ✅ Site points back to the repo (1 pt)

LANGUAGES AND AUTOMATA II

HOMEWORK #3 PARTIAL ANSWERS

1. In Java
   a. `x+++-y` is **not a compile-time error** (it parses as `(x++)+(-y)`)
   b. `x---+y` is **not a compile-time error** (it parses as `(x--)-(+y)`)
   c. Incrementing a read-only variable is a **static semantic error**, as there is no way in Java to tell whether a variable is read-only from the grammar alone.
   d. Code in class C accessing a private field from class D is a **static semantic error** since access attributes are not attached to identifiers in the grammar.
   e. Using an uninitialized variable is a **static semantic error** since not being initialized cannot be known from the grammar alone, and Java, unlike C, does not allow this.
   f. Dereferencing a null reference is **not a compile-time error**. There is a chance a warning can occur if the compiler can prove the reference is always null. It's not even a dynamic semantic error, since a deference just throws an exception that can be caught.
   g. `null instanceof C` is **not a compile-time error**, but it's kind of silly to write because it always returns `false`.
   h. `!!x` is **not a compile-time error** (since, remember, the context of this problem is to assume the types make as much sense as they can).
   i. `x > y > z` is a **static semantic error** since the expression is allowed by the grammar, but the `>` operator is not allowed on booleans!
   j. `if (a instanceof Dog d) {...}` is **not a compile-time error**. The `...` was not meant to be taken literally.
   k. `var s = """This is weird""";` is a **syntax error** because an opening triple-quote must end a line.
   l. `switch = 200;` is a **syntax error** because `switch` is a reserved word.
   m. `x = switch (e) {case 1->5; default->8;};` is **not a compile-time error** as modern Java allows switch expressions.

2. JavaScript disallows this (at least in strict mode) because you cannot redeclare an identifier in a scope. Rust does allow shadowing, but in this case, it technically rejects because the first `x` is never used. The important thing to say is that in Rust, there are two variables here, not one.

3. In Java, a `private` member of an object is visible only in fields and methods within the same class as the object. In Ruby, a private function is one that may not be called with an explicit receiver; instead, the receiver is always implicitly `self`.

   The important difference is that if we have two objects, `x` and `y` of class `C`, say, then in Java, a method called on `x` *can* call a private method on `y`, because the two objects have the same class. However, in Ruby, this can't happen: a method on `x` can only call private methods on `x` itself! So Ruby `private` access is pretty much object-based, while in Java it's class-based.

   Example in Java:

```
jshell> class C {
... private String f() {return "hi";}
... String g(C other) {return other.f();}
...}
|  created class C

jshell> var a = new C();
a ==> C@77caeb3e

jshell> var b = new C();
b ==> C@3bfdc050

jshell> b.g(a)
$4 ==> "hi"
```

   Note that in Java, you can change a private field of an object *through a different object*. This is very permissive! Not super secure.

   This cannot be written in Ruby at all because (1) there is *no way to refer to fields in other objects at all*, and the lack of a receiver on a private method means there is no possible way to call a private method on a different object other than the current one (`self`).

   In case you are interested is an example of how we would use `private` in Ruby:

cylinder.rb

```ruby
# Illustration of private methods in Ruby. Private methods cannot have an
# explicit receiver so they always apply to the existing receiver. In other
# words, they are private to the current object.
```

```ruby
class Cylinder
  def initialize(r = 1, h = 1)
    @r = r
    @h = h
  end
  def volume
    capArea * @h
  end
  def surfaceArea
    2 * capArea + sideArea
  end

  private

  def capArea
    Math::PI * @r * @r
  end
  def sideArea
    Math::PI * 2.0 * @r * @h
  end
end

c = Cylinder.new(10, 50)
puts c.volume
puts c.surfaceArea
```

Now it *is* true that if you subclass a class in Ruby, the subclass will have those private methods, too. This is different from Java, where private methods are not inherited. However, the "inherited" private methods can only work on a single object. Ruby never allows you to change a private field in one class by calling a method on a different class.

4. Yes! Suppose `g` were a function which modified the global variable `a` as a side effect. Then the expression `f(a,g(b))` could yield different results depending on which order the arguments were evaluated. (I am hoping your answer shows an exact case!)

5. Carlos used to prohibit recursive structs because structs are value types and are (at least conceptually) stored directly rather than being allocated on the heap. Therefore a "recursive" struct would have infinite size. So when analyzing a struct type T, the compiler will look for any fields that have type T, and if it finds any, will generate an error. The search itself recurses into

any fields of the struct which happen themselves to be structs. The code for this, which used to be on GitHub, is:

```
function includesAsField(structType, type) {
  // Whether the struct type has a field of type type, directly or indirectly
  return structType.fields.some(
    field =>
      field.type === type ||
      (field.type?.kind === "StructType" && includesAsField(field.type, type))
  )
}

function mustNotBeSelfContaining(structType, at) {
  const containsSelf = includesAsField(structType, structType)
  must(!containsSelf, "Struct type must not be self-containing", at)
}
```

In later versions of Carlos, this restriction was relaxed, allowing recursive struct **type definitions** to be written, for example:

```
struct Useless {
  useless: Useless
}
```

We ended up allowing this because it turns out not to matter because you cannot create an instance of this type anyway. Try it! (If you can prove me wrong, you will get 20 points of extra credit.)

6. The minimum value of a (non-empty) sequence is: (1) the first element, if the sequence has length one, or (2) the minium of the first element and the minimum value of the rest of the sequence. In pseudocode you might write:

```
function min_element(a)
    if len(a) == 1 then
        return a[0]
    else
        return min(a[0], min_element(a[1..]))
    end
end
```

However, this is *not* tail recursive, and is not an acceptable solution. To make it tail recursive you need the trick where your recursive function accepts a list and the current minimum so far. Here are some solutions to the tail-recursive part. In most cases these would need to be wrapped in a function which did not take in the "accumulator" min-so-far variable.



```python
def minimum(a, min_so_far = math.inf):
    if not a:
        return min_so_far
    return minimum(a[1:], min(a[0], min_so_far))
```



```c
double minimum(double *array, int len, double min_so_far) {
    if (len == 0) {
        return min_so_far;
    }
    double new_min = array[0] < min_so_far ? array[0] : min_so_far;
    return minimum(array+1, len-1, new_min);
}
```



```javascript
function minimum(array, minSoFar = Infinity) {
    if (array.length === 0) return minSoFar
    return minimum(array.slice(1), Math.min(a[0], minSoFar))
}
```



```go
// Thanks to ChatGPT for this
func minimum(array []float64, minSoFar float64) float64 {
    if len(array) == 0 {
        return minSoFar
    } else {
        newMin := minSoFar
        if minSoFar < array[0] {
            newMin = array[0]
        }
        return minimum(array[1:], newMin)
    }
}
```



```erlang
minimum([], Min) -> Min;
minimum([Head | Tail], Min) -> minimum(Tail, min(Head, Min)).
```

```rust
fn minimum(arr: &[f64], min_so_far: f64) -> f64 {
    match arr {
        [] => min_so_far,
```

```
            [head, tail @ ..] =>
                minimum(tail, if *head < min_so_far { *head } else { min_so_far })
        }
    }
```

7. The second friend's code is passing `undefined` to the callback of `setTimeout`. This is an example where delayed execution is called for, so to fix it, you need to replace `update(i)` with `()=>update(i)`. There's another way, too. You can invoke `setTimeout` with a third argument: `setTimeout(1000, update, i)`. That's a bit harder to read, but it does work!

8. Using SonarLint and PMD I found:
    - The file is not in a named package.
    - Should use a utility class (private constructor) since everything is static.
    - Should use interface Map for the variable declaration, not HashMap.
    - Since HashMap was used, the right hand side should have used a diamond.
    - Empty constructor should be documented.
    - Public constructor should be hidden.
    - Should use a constant instead of a method for `zero`.

    FindBugs and FindSecBugs might find more!

L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #4 Due: 2025-04-22

Learning Objectives

In this assignment you will demonstrate:

- ✅ A familiarity with the basics of the four major computation theories
- ✅ A decent understanding of formal language theory and its importance within computer science
- ✅ Knowledge of the foundations of Automata Theory and Computation Theory, including the Chomsky Hierarchy, the difference between decidability and acceptability.
- ✅ The ability to craft Turing Machines for various programming and decision problems
- ✅ Working knowledge of register machines both as a model of computation and as a target of compilation
- ✅ An understanding of intermediate languages and virtual machines
- ✅ The ability to write the back end of a compiler, targeting a virtual machine or a high level language

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Do any of the assigned readings and watchings from the first three assignments that you did not get to.
- ✅ Work through the course notes on How to Write a Compiler, Theories of Computer Science, Language Theory, Automata Theory, Turing Machines, Register Machines, and Computability Theory.

- ✅ Read Ch. 6–8 in [Bernhardt]
- ✅ Read Ch. 4 of [Sipser], and skim Ch. 5–6
- ✅ Read Ch. 7 of [Mogenson]
- ✅ To the best of your ability, as time allows, skim Turing's most famous paper
- ✅ Skim the Wikipedia articles on Turing Machines and Undecidability
- ✅ Skim the Wikipedia articles on Transpilers, Intermediate Language, Virtual Machine, LLVM, Java Virtual Machine, C--, WebAssembly, Single Static Assignment

Submission Instructions

As usual, turn in a single submission for your entire group via BrightSpace. Remember to not simply partition the work among team members, as this reduces your learning.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project

Exercises

1. In your own words, write one sentence for each of the four major theories of computation, conveying its central question and its areas of concern. Write as if your job depended on clarity, accuracy, and solid English writing skills. If you look to an AI assistant for help, do not just copy what the bot says; but feel free to try so see if it can even get close on this one. The scope of the four theories are kind of fuzzy, so stick with the definitions you've heard in class rather than the bot's training set.

2. Given $L_1 = \{0, 011, 10\}$ and $L_2 = \{10, 1\}$. What are:
   a. $L_1 \cup L_2$
   b. $L_1 \cap L_2$
   c. $L_1 L_2$

d. $L_2{}^*$

3. Give grammars for the following languages (using the notation from class):
   a. The empty language
   b. $\{0^i 1^j 2^k \mid i = j \vee j = k\}$
   c. $\{w \in \{0,1\}^* \mid w \text{ does not contain the substring } 000\}$
   d. $\{w \in \{a,b\}^* \mid w \text{ has twice as many } a\text{'s as } b\text{'s}\}$
   e. $\{a^n b^n a^n b^n \mid n \geq 0\}$

   (Note that not all of these are expressible with Ohm.)

4. Here's another look at the grammar for floating-point numerals (using single-letter variables for compactness):

   $n \longrightarrow d^+ \ f? \ e?$
   $f \longrightarrow$ `"."` d^+
   $e \longrightarrow$ (`"E"`|`"e"`) (`"+"`|`"-"`)? d^+
   $d \longrightarrow$ `"0"..."9"`

   Give the (V, Σ, R, S)-definition of this grammar. (Note this means you will have to desugar the rules with `|`, `?`, and `+`.)

5. Give Turing Machines that recognize the following languages. If any of the languages below are Type-3, you may (and are encouraged to) give a FA in lieu of a TM recognizer, if the FA is simpler.
   a. $\{w \in \{a,b\}* \mid w \text{ ends with } abb\}$
   b. $\{w \in \{a,b\}^* \mid \#_a(w) = \#_b(w)\}$ (same number of a's as b's)
   c. $\{w \in \{a,b\}* \mid w \text{ alternates } a\text{'s and } b\text{'s}\}$
   d. $\{a^n b^n a^n b^n \mid n \geq 0\}$

6. Give Turing Machines that compute the following functions, where the input and output are binary numerals.
   a. $\lambda n.\ 2n + 2$
   b. one's complement
   c. The function described in Python as `lambda n: str(n)[1:-1]`

7. For the JavaScript/Python expression `5 * 3 - 1 ** 3`,
   a. Show a 3AC machine program to evaluate this expression, leaving the result in r_0

b. Show a Stack machine program to evaluate this expression, leaving the result on the top of the stack.

8. Characterize each of the following languages as either (a) regular, (b) context-free but not regular, (c) recursive but not context-free, (d) recursively enumerable but not recursive, or (e) not even recursively enumerable.
   a. $\{a^i b^j c^k \mid i > j > k\}$
   b. $\{a^i b^j c^k \mid i > j \wedge k \leq i - j\}$
   c. $\{\langle M \rangle \cdot w \mid M \text{ accepts } w\}$
   d. $\{G \mid G \text{ is context-free} \wedge L(G) = \varnothing\}$
   e. $\{a, b\}^* \{b\}^+$
   f. $\{\langle M \rangle \mid M \text{ does not halt }\}$
   g. $\{w \mid w \text{ is a decimal numeral divisible by 7}\}$
   h. $\{www \mid w \text{ is a string over the Unicode alphabet}\}$

For Your Project

Continue your compiler project in the public GitHub repository you created in the first assignment. You will be expanding your repo to have the following:

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json
├── docs
|   └── ...
├── examples
|   └── ...
├── src
|   ├── (yourlanguagename).js
|   ├── (yourlanguagename).ohm
|   ├── compiler.js
|   ├── core.js
|   ├── analyzer.js
|   ├── optimizer.js        -- updated as below
|   └── generator.js        -- flesh this out!
└── test
    ├── compiler.test.js
    ├── grammar.test.js
```

```
├── analyzer.test.js
└── generator.test.js    -- new!
```

In this assignment you will be completing the following tasks:

- Complete the **code generator** portion of your compiler. You may output either C, C--, Java (source or JVM), LLVM, JavaScript, WebAssembly, the assembly language in the textbook, or code for any VM or IR you like (even one you create yourself), provided you check with me first. Implement the generator in src/generator.js and the tests in test/generator.test.js.
- Update the file src/optimizer.js to basically be a big no-op. Rewrite it like so:

```
export default function optimize(node) {
  return node
}
```

- Keep improving your README.
- Tidy up any issues found in the grammar and analyzer portions of your compiler (or any other component).
- Do any upgrades as is necessary for your companion website on GitHub pages.

Grading Rubric

To help you get a good score, here are all the things you will be graded on.

- Exercises (80 pts)
  - ✅ Exercise 1 (10 pts)
  - ✅ Exercise 2 (10 pts)
  - ✅ Exercise 3 (10 pts)
  - ✅ Exercise 4 (10 pts)
  - ✅ Exercise 5 (10 pts)
  - ✅ Exercise 6 (10 pts)
  - ✅ Exercise 7 (10 pts)
  - ✅ Exercise 8 (10 pts)
- Compiler Project (20 pts)
  - ✅ All tests pass (2 pts)
  - ✅ Analyzer is complete (1 pt)
  - ✅ Generator is well structured (2 pts)

- ✅ Generator is complete (5 pts)
- ✅ You have “interesting” code generation (2 pts)
- ✅ You have a large number of tests for the code generator (2 pts)
- ✅ The generated code is readable (1 pt)
- ✅ Examples of generated code is on the README or companion site (2 pt)
- ✅ Test coverage is 100% (2 pts)
- ✅ Can run analyzer and generator from command line (1 pt)

Make sure to make all corrections on your project and project companion site from the previous assignments.

LANGUAGES AND AUTOMATA II

HOMEWORK #4 PARTIAL ANSWERS

1. Language Theory explores with how computations are **expressed** and is concerned with syntax, semantics, and pragmatics. Automata Theory explores how computations are **carried out by machine** and is concerned with developing formal models of computation that we can reason about. Computability theory explores the **limits of computation** and is concerned with what can and cannot be computed and why. Complexity Theory explores the **resources required** for certain computations and is concerned with how efficiently (in terms of time and space) problems can be computed.

2. a. $L_1 \cup L_2 = \{0, 011, 10, 1\}$
   b. $L_1 \cap L_2 = \{10\}$
   c. $L_1 L_2 = \{010, 01, 01110, 0111, 1010, 101\}$
   d. $L_2{}^* = \{\varepsilon, 10, 1, 1010, 101, 110, 11, 101010, 10101, \ldots\}$

3. a. The empty language

There are NO rules in this grammar

   The empty language, the language of no strings, is absolutely NOT the same as the language containing only the empty string.

   While the best answer is that the grammar has a empty set of rules, it is also acceptable to create a one-rule grammar of the form $s \longrightarrow s$. In this grammar, you can never derive anything, so, it does work, but it's not a great answer.

   b. $\{0^i 1^j 2^k \mid i = j \vee j = k\}$

s	$\longrightarrow$	x "2"* | "0"* y
x	$\longrightarrow$	ε | "0" x "1"
y	$\longrightarrow$	ε | "1" y "2"

c. $\{w \in \{0,1\}^* \mid w$ **does not contain the substring 000**$\}$

s	$\longrightarrow$	$(x$ `"1"`$)^*\ x$
x	$\longrightarrow$	$\varepsilon \mid$ `"0"` $\mid$ `"00"`

This generates an arbitrary number of "chunks" of at most two zeros separated by 1s

d. $\{w \in \{a,b\}^* \mid w$ **has twice as many** a**'s as** b**'s**$\}$

s	$\longrightarrow$	x^*
x	$\longrightarrow$	`"a"` x `"a"` x `"b"` $\mid$ `"a"` x `"b"` x `"a"` $\mid$ `"b"` x `"a"` x `"a"`

This generates zero or more "chunks" each of which has twice as many a's as b's. There are three ways to form a chunk: (1) `a---a---b`, (2) `a---b---a`, and (3) `b---a---a`.

e. $\{a^n b^n a^n b^n \mid n \geq 0\}$

s	$\longrightarrow$	(`"a"` 0 `"b"` 1 `"a"` 2 `"b"`)?	Place marks between blocks
`"a"` 0	$\longrightarrow$	`"aa"` 0 x	New a's at the beginning
x `"b"`	$\longrightarrow$	`"b"` x	Move x's to the right of b's
x 1	$\longrightarrow$	`"b"` 1 x	When x hits the 1, make a new b
x `"a"`	$\longrightarrow$	`"a"` x	Move x past the second batch of a's
x 2	$\longrightarrow$	`"a"` 2 `"b"`	When x hits the 2, generate an ab on the right
0	$\longrightarrow$	ε	Drop the marks at any time
1	$\longrightarrow$	ε	
2	$\longrightarrow$	ε	

There is probably a much better way.

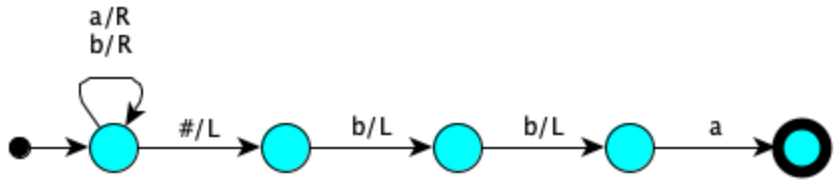
4. To show the grammar in the standard (V, Σ, R, S) form we have to get rid of all the wonderful convenient operators and introduce new variables. Here I added s to stand for "sequence of one or more digits." Also, since e appears in the language but was also being used as a variable, I've changed the e variable to x (still stands for "exponent part").

$$
\begin{(}
\quad \{n, s, f, x, d\},\\
\quad \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., E, e, +, -\},\\
\quad \{\\
\qquad (n, sfx),\\
\qquad (s, d), (s, ds),\\
\qquad (f, \varepsilon), (f, .s),\\
\qquad (x, \varepsilon), (x, Es), (xE + s), (x, E - s), (x, es), (x, e + s), (x, e - s),\\
\qquad (d, 0), (d, 1), (d, 2), (d, 3), (d, 4), (d, 5), (d, 6), (d, 7), (d, 8), (d, 9)\\
\quad \},\\
\quad n\\
\end{)}
$$

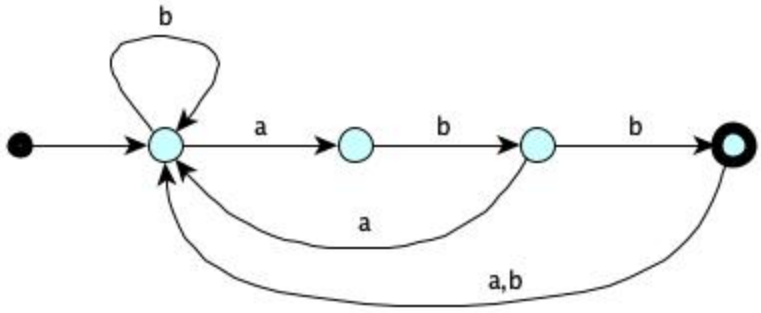
5. Turing Machine recognizers:

   a. $\{w \in \{a, b\}* \mid w \text{ ends with } abb\}$

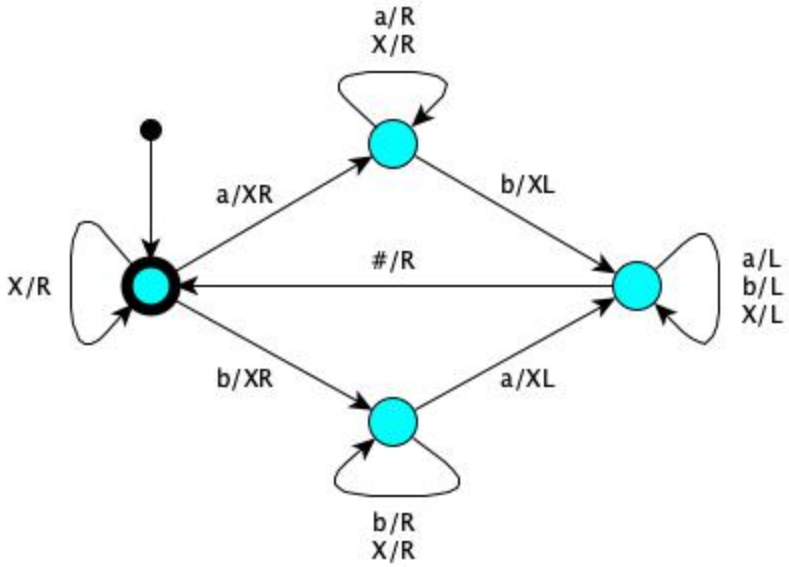
   The most direct way is to move to the end then go backwards through the last 3 symbols:



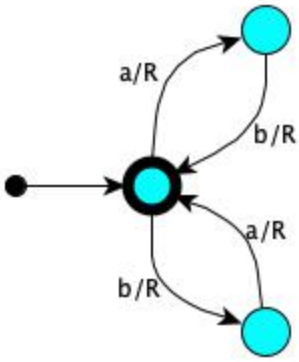
   You can also do this as a FA:



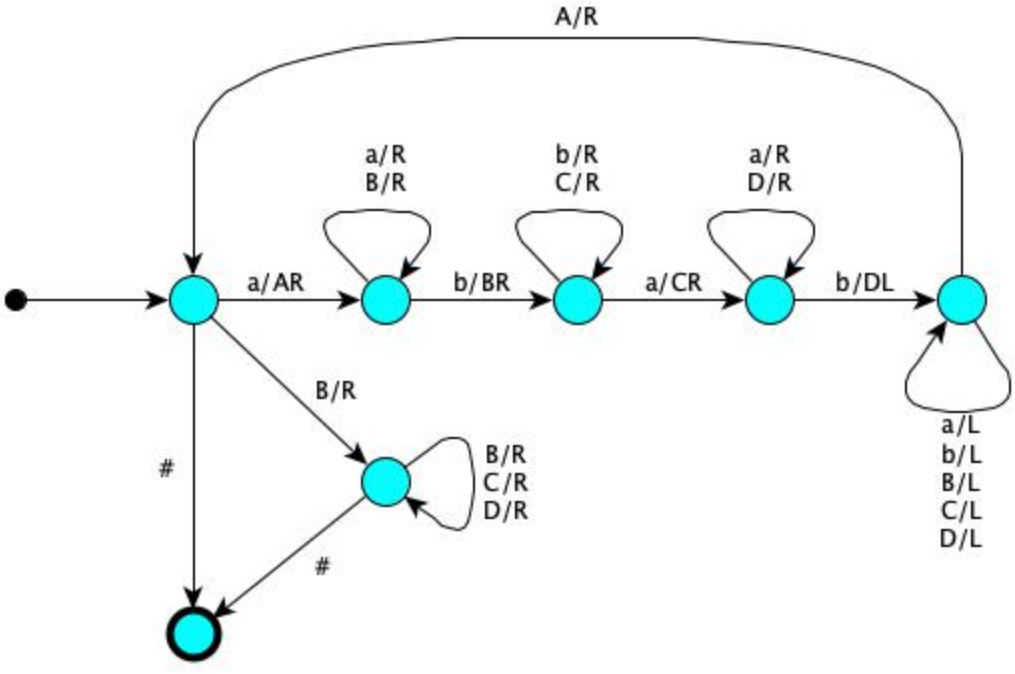
   b. $\{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ (same number of a's as b's)



c. $\{w \in \{a, b\}* \mid w \text{ alternates } a\text{'s and } b\text{'s}\}$



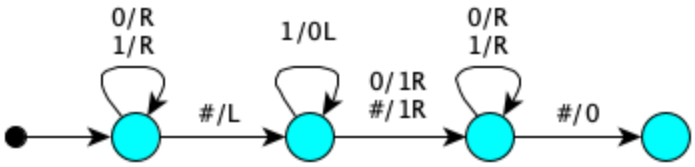
d. $\{a^n b^n a^n b^n \mid n \geq 0\}$



6. Turing Machine transducers:

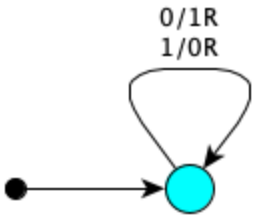
   a. $\lambda n.\, 2n + 2$

      You can first increment, then double:

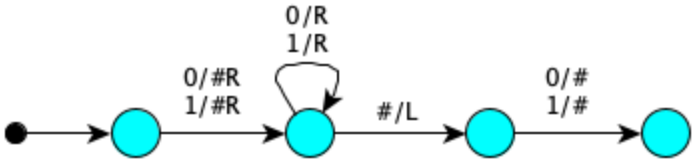


      There is another way which is to first double, then add two by starting the increment process not at the original rightmost symbol but at the one before it. Try it!

   b. One's complement. Just flip all the bits as you move right.



   c. The function described in Python as `lambda n: str(n)[1:-1]`. This is done by "erasing" the first and last symbols, i.e., writing over them with blanks.



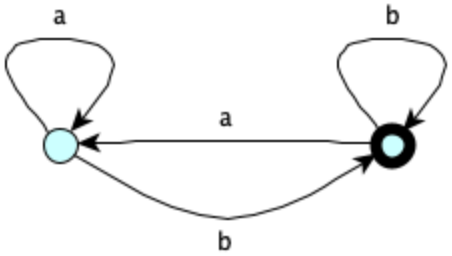
7. The expression `5 * 3 - 1 ** 3` can be evaluated on a 3AC machine like this:

```
mul r1, 5, 3
pow r2, 1, 3
sub r0, r1, r2
```

   and on a 0AC (stack) machine like this:

```
load 5
load 3
mul
load 1
load 3
pow
sub
```

8. Here (a) = regular, (b) = context-free but not regular, (c) = recursive but not context-free, (d) = recursively enumerable but not recursive, and (e) = not even recursively enumerable.
   a. $\{a^i b^j c^k \mid i > j > k\}$ (c) R
      This is not CF because when recognizing on a PDA, you push when you see an a, then pop when seeing b's. You know there should still be a's left after popping, but the number of marks remaining on the stack is only $i - j$. You don't know how big i and j were, so there is no way to check k. It is obviously recursive since a trivial Python program can easily decide it by just counting.
   b. $\{a^i b^j c^k \mid i > j \wedge k \leq i - j\}$ (b) CF
      This is CF because when recognizing on a PDA, you push when you see an a, then pop when seeing b's. You know there should still be a's left after popping, and this number $i - j$. Pop when you see c's and insure there is always something to pop. Also you can see the language is not regular since you have to remember the value of i, which is arbitrarily large.
   c. $\{\langle M \rangle \cdot w \mid M \text{ accepts } w\}$ (d) RE
      This is the classic language-acceptance problem whose proof that is r.e. but not recursive can be found in Sipser.
   d. $\{G \mid G \text{ is context-free} \wedge L(G) = \varnothing\}$ (d) RE
      Rice's Theorem applies here.
   e. $\{a, b\}^*\{b\}^+$ (a) REG
      This FA accepts it:



   f. $\{\langle M \rangle \mid M \text{ does not halt }\}$ (e) NOT RE
      A well-known result. This was on the course notes for computability!

g. **$\{w \mid w$ is a decimal numeral divisible by 7$\}$** (a) REG

A Finite Automata can track the remainder as digits are added.

h. **$\{www \mid w$ is a string over the Unicode alphabet$\}$** (c) R

Not context free since the ww language is not even context free! But definitely recursive as you can split the string into three parts and just check in a way you will always finish.

L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #5 Due: 2025-05-06

Learning Objectives

In this assignment you will demonstrate:

- ✅ The ability to write regular expressions
- ✅ The ability to perform a problem reduction to demonstrate undecidability
- ✅ An understanding of the major optimizations from compiler history (especially the Frances Allen ones)
- ✅ The ability to write optimizations in a compiler
- ✅ Excellence in growing and maintaining a large Node.js project

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Do any of the assigned readings and watchings from the first four assignments that you did not get to.
- ✅ Work through the course notes on How to Write a Compiler, Regular Expressions, Intermediate Representations, and Code Optimization.
- ✅ Read Ch. 9 in [Bernhardt]
- ✅ Read Ch. 7 of [Sipser], and skim Ch. 8-10
- ✅ Skim Ch. 10-12 of [Mogenson] (paying attention 11-12)
- ✅ Browse the vast amount of material at regular-expressions.info. Play around with Regex 101 and Regex Golf. Mastering regular expressions is something that should be expected of you at this stage of your career. To see some examples of regexes in the wild, check out this overview of credit card validation.

- ✅ Browse the Wikipedia article on Program Optimization
- ✅ Study this "Optimization Killers" article. You'll be glad you did. There are so many good things in here you can impress your friends (and future interviewers) with!
- ✅ Browse this great presentation by Graydon Hoare. You don't have to understand every slide, but jot down notes from some of the big ideas. If it gets too technical, don't worry, just move on.
- ✅ Read Frances Allen's classic A Catalogue of Optimizing Transformations. This work followed two of her previous papers where she created new frameworks and algorithms and structures for performing and implementing static program analysis. You may also wish to read this remembrance from IBM (where she was a fellow), or any number of bibliographies on her life and work.

The following readings are optional in the sense that no direct assessment will be based on them. However, as this course marks your deepest dive yet into computing theory and history, you'd be wise to familiarize yourself with the following:

- ✅ Alan Turing's Forgotten Ideas
- ✅ Ada and the First Computer
- ✅ Decoding an Ancient Computer
- ✅ The Origins of Computing

Submission Instructions

As usual, turn in a single submission for your entire group via BrightSpace. Remember to not simply partition the work among team members, as this reduces your learning.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project

Exercises

1. Write JavaScript regular expressions for the following. Please take advantage of character classes, lookarounds, and backreferences where they apply.
   a. Canadian Postal Codes (make sure to prohibit D, F, I, O, Q, U)
   b. Legal Visa® Card Numbers, *ignoring the Luhn checksums*, i.e., accept `4` + 15 digits or `4` + 12 digits.
   c. Legal MasterCard® Numbers, *ignoring the Luhn checksums*, i.e., accept `51–55` + 14 digits or `2221–2720` + 12 digits.
   d. Strings of Basic Latin letters **except** those strings that are exactly three letters ending with two Latin letter o's, of any case.
   e. Binary numerals divisible by 16.
   f. Decimal numerals in the range 8 through 32, inclusive.
   g. All strings of Unicode letters, except `python`, `pycharm`, or `pyc`.
   h. Floating point constants that are allowed to have an empty fractional part, but whose exponent part is required and can have no more than three digits in the exponent part
   i. Palindromes over the letters a, b, and c, of length 2, 3, 5, or 8
   j. Python string literals. Don't get too fancy here—just translate what you see in the Python Reference linked above into Ohm notation.

   Structure your application and its unit tests as follows. The test file *test/regex_exercises.test.js* should be:

```
import { describe, it } from "node:test"
import assert from "assert"
import { matches } from "../regex_exercises.js"

const testFixture = {
  canadianPostalCode: {
    good: ["A7X 2P8", "P8E 4R2", "K1V 9P2", "Y3J 5C0"],
    bad: ["A7X   9B2", "C7E 9U2", "", "Dog", "K1V\t9P2", " A7X 2P8", "A7X 2P8 "]
  },
  visa: {
    good: ["4128976567772613", "4089655522138888", "4098562516243"],
    bad: [
      "43333",
      "423462387462837468 23",
      "7687777777263211",
      "foo",
      "π",
      "4128976567772613 ",
```

```
  ],
},
masterCard: {
  good: [
    "5100000000000000",
    "5294837679998888",
    "5309888182838282",
    "5599999999999999",
    "2221000000000000",
    "2720999999999999",
    "2578930481258783",
    "2230000000000000",
  ],
  bad: [
    "5763777373890002",
    "513988843211541",
    "51398884321108541",
    "",
    "OH",
    "5432333xxxxxxxxx",
  ],
},
notThreeEndingInOO: {
  good: ["", "fog", "Tho", "one", "a", "ab", "food"],
  bad: ["fOo", "gOO", "HoO", "zoo", "MOO", "123", "A15"],
},
divisibleBy16: {
  good: ["0", "00", "000", "00000", "00000", "000000", "00000000", "1101000000
  bad: ["1", "00000000100", "1000000001", "dog0000000"],
},
eightThroughThirtyTwo: {
  good: Array(25)
    .fill(0)
    .map((x, i) => i + 8),
  bad: ["1", "0", "00003", "dog", "", "361", "90", "7", "-11"],
},
notPythonPycharmPyc: {
  good: ["", "pythons", "pycs", "PYC", "apycharm", "zpyc", "dog", "pythonpyc"]
  bad: ["python", "pycharm", "pyc"],
},
restrictedFloats: {
```

```
  good: ["1e0", "235e9", "1.0e1", "1.0e+122", "55e20"],
  bad: ["3.5E9999", "2.355e-9991", "1e2210"],
},
palindromes2358: {
  good: [
    "aa",
    "bb",
    "cc",
    "aaa",
    "aba",
    "aca",
    "bab",
    "bbb",
    "ababa",
    "abcba",
    "aaaaaaaa",
    "abaaaaba",
    "cbcbbcbc",
    "caaaaaac",
  ],
  bad: ["", "a", "ab", "abc", "abbbb", "cbcbcbcb"],
},
pythonStringLiterals: {
  good: String.raw```
    ""
    'hello'
    "world"
    'a\'b'
    "a\"b"
    '\n'
    "a\tb"
    f'\u'
    """abc"""
    '''a''"''"'''
    """abc\xdef"""
    '''abc\$def'''
    '''abc\''''`
    .split("\n")
    .map((s) => s.trim()),
  bad: String.raw`
    'hello"
```

```
      "world'
      'a'b'
      "a"b"
      'a''
      "x""
      """""""""
      frr"abc"
      'a\'
      '''abc''''
      """`
      .split("\n")
      .map((s) => s.trim()),
  },
}

for (let name of Object.keys(testFixture)) {
  describe(`when matching ${name}`, () => {
    for (let s of testFixture[name].good) {
      it(`accepts ${s}`, () => {
        assert.ok(matches(name, s))
      })
    }
    for (let s of testFixture[name].bad) {
      it(`rejects ${s}`, () => {
        assert.ok(!matches(name, s))
      })
    }
  })
}
```

The file *regex_exercises.js* can be structured like so:

```
const regexes = {
  canadianPostalCode: /^ ........ $/,
  visa: /^ ........ $/,
  masterCard: /^ ........ $/,
  notThreeEndingInOO:  /^ ........ $/iu,
  divisibleBy16:  /^ ........ $/,
  eightThroughThirtyTwo:  /^ ........ $/,
  notPythonPycharmPyc:  /^ ........ $/u,
```

```
  restrictedFloats:  /^ ........ $/i,
  palindromes2358: /^ ........ $/,
  pythonStringLiterals: /^ ........ $/,
}

export function matches(name, string) {
  return regexes[name].test(string)
}
```

2. For the following C function:

```
int f(const int n) {
  return n % 2 == 0 ? n / 2 : 3 * n + 1;
}
```

   give both a highly optimized WebAssembly translation and a highly optimized x86-64 translation. You do not have to do this by hand; instead, use the Compiler Explorer, and set the optimization to `-O3`. Include comments in the translated code.

3. (Here's an exercise which logically belongs in the previous homework assignment, but sometimes it's best to space things out a bit.) Show the language $\{\langle M_1 \rangle \langle M_2 \rangle \mid L(M_1) = L(M_2)\}$ is undecidable, using a rigorous reduction argument.

For Your Project

Add optimizations to your compiler project. You do not have to do too many, but the few you do should be non-trivial. Because everyone in class is compiling a somewhat different language, you must check with me *early* regarding which optimizations are appropriate for your project. Don't worry, we will negotiate the set of optimizations that you are responsible for, so there should be no surprises when you get your grade.

Your project should end up with a project structured as follows:

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json        -- (optional, you don't have to have one)
├── docs
```

```
|   └── ...                    -- logo, companion website, etc.
├── examples
|   └── ...                    -- lots of example programs
├── src
|   ├── (yourlanguagename).js
|   ├── (yourlanguagename).ohm
|   ├── compiler.js
|   ├── core.js
|   ├── analyzer.js
|   ├── optimizer.js           -- implement this for this assignment
|   └── generator.js
└── test
    ├── compiler.test.js
    ├── grammar.test.js
    ├── analyzer.test.js
    ├── optimizer.test.js      -- implement this for this assignment
    └── generator.test.js
```

The most important things to keep in mind as you wrap up your project are:

- 100% test coverage! Hold yourself to this goal. Be proud!
- Have your README and companion website completely up to date with your language. If there are parts of your language documented in the README or companion site but not yet implemented, you must mark your README or companion site with a "Not yet implemented" warning. If you have code in your compiler that is *not* documented in your README or companion site, that will result in a loss of points, as this is a situation that should not happen.
- Make sure there are no additional, unnecessary files in your repo. A pristine repo is a happy repo!
- Ensure all of the code has been Prettied. Your repo is public and you want to show your best face to the world!

Presentation!

During the last class session, **present, as your entire group, a lightning talk introducing (and showing off!) your language to the rest of the class**. Be personal, be funny, and be brief! You may use slides, video, and even music. If you are using slides, don't use too many, and make sure they are colorful...with images, *and please put these in your project repository*. If your language has a theme (pirates, horror, uwu, vikings, fashion, sci-fi, anime, social policy, medical, whatever), you can dress up (but avoid stereotypes and be respectful).

For tips on how to do a good lightning talk, see here, here, and here. What's a lightning talk? Wikipedia can tell you.

Grading Rubric

To help you get a good score on the remaining 80 points, here are all the things you will be graded on.

- Exercises (25 pts)
  - ✅ Exercise 1 (15 pts)
  - ✅ Exercise 2 (5 pts)
  - ✅ Exercise 3 (5 pts)
- Finished Compiler Project (45 pts)
  - ✅ Optimizer is strong
  - ✅ Project clones and runs without crashing
  - ✅ package.json is complete (author names, type=module, test script, etc.)
  - ✅ All tests pass
  - ✅ 100% test coverage
  - ✅ Command line operations that run the parser, the analyzer, and the generator separately all work
  - ✅ Can analyze, optimize, and generate code for Hello World
  - ✅ Can analyze, optimize, and generate code for a medium size program
  - ✅ Can analyze, optimize, and generate code for a really large program
  - ✅ Errors are detected and reported cleanly when run from the command line
  - ✅ Your language reached the point of being “complex enough.”
  - ✅ Code is nice, clean, well-organized, professional
  - ✅ Code has been prettied
  - ✅ Analyzer, optimizer, and code generator are all complete
  - ✅ README page is complete, useful, and accurate
  - ✅ Companion website is complete, useful, and accurate
  - ✅ README page OR companion website has a link to the grammar, making it easy to lookup
  - ✅ You have a lot of really good examples in your examples directory
  - ✅ Your logo looks good
  - ✅ Project commit log shows that you've been working all semester
  - ✅ Optional but strongly encouraged: You included something “fun” in the compiler—perhaps an easter egg (document the easter egg in the companion website)
- Presentation (30 pts)
  - ✅ You introduced yourselves
  - ✅ You introduced your language
  - ✅ You told the story of your language in a fun and engaging way
  - ✅ Slides or other visual aids are compelling
  - ✅ If you do have slides (you should) they show some interesting snippets of your language
  - ✅ The talk is not too short or too long

- ✅ The talk has some “structure” (prove to me you read the materials I linked to about how to make a
  lightning talk effective).
- ✅ Intro and outro were compelling, exciting, and memorable!

Also, as an audience member, you will be expected to be attentive, respectful, and encouraging, and celebrate and applaud your classmates when they present.

> ***Fairness in Team Grading***
>
> *Finally, I will be interviewing each team to gauge the contributions of each team member. The old ”I participated in meetings but we always committed from so-and-so’s computer” isn’t really acceptable.*